

Script: Day 1 of Building an AI Business From Scratch (and the System I Wish I'd Had on Day Zero)

voice_reference:	../../Resources/voice_reference/nate_herk_voice_reference.md
prep_notes:	../../Resources/scrape_pipeline_prep_notes.md
plan:	/Users/danieldanut/.claude/plans/amazing-one-youtuber-i-m-streamed-anchor.md
script_type:	Hybrid (TIGHT intro + LOOSE demo + TIGHT outro)
hook_type:	Personal-Moment / Build-in-Public
episode:	1
series:	Building an AI Business From Scratch
target_words:	1,800–2,100
target_length:	12–15 min @ 150 wpm
draft_date:	2026-05-09
status:	draft v1 — ready for read-through

Production legend

**[brackets]* = visual cue / on-screen note*

Format tag at top of each section: TH (talking head, pre-recorded) or LIVE (screen share + terminal)

Beat counts after each section header are spoken word targets

[HOOK] — TH, pre-recorded — ~52 words

[Plain background. Daniel direct to camera. No B-roll.]

I'm building an AI business from scratch — and I'm putting every step of it on this channel. The good, the bad, the parts I get wrong. This is day one. And before you ask — yes, I already did the weird thing of transcribing over two thousand videos before letting myself hit upload.

Let me show you why.

[PROMISE] — TH, pre-recorded — ~48 words

By the end of this video, you'll have a free system that pulls every single video from any YouTube channel and turns it into clean text in a few minutes. You'll know exactly what I'm doing with all that data. And you'll know exactly what I'm building toward.

[CREDENTIAL] — TH, pre-recorded — ~76 words

[Same shot as hook. Slow down here.]

Real quick — I don't have years of YouTube experience, I haven't built a successful AI business yet, and I'm not going to pretend I have. That's actually the whole point of this channel. You're going to watch me figure it

out — including the parts I get wrong. What I had on day zero was a basic Windows laptop, a couple of weeks, and a habit of going way too deep on research. That's where I started.

[PIPELINE DEMO] — LIVE (screen share + terminal) + occasional TH cutaways — ~680 words

[Cut to screen share. Daniel narrates over the screen. Terminal + browser visible.]

Okay, so here's the punchline that surprised me when I was building this — I never had to actually transcribe anything. YouTube already transcribed every video on its platform. Auto-captions. The hard part isn't the transcribing. The hard part is downloading them in bulk. And once you crack that, you can have transcripts of two thousand videos sitting on your laptop in like ten minutes.

[Open YouTube channel — Nate Herk's Videos tab.]

Step one — get the video links. Go to any YouTube channel's Videos tab. Scroll all the way down to load every video — yes, you actually have to scroll, YouTube lazy-loads. Then hit Ctrl+A to select everything on the page, Ctrl+C to copy.

[New Google Sheets tab. Click A2. Paste.]

Now open a fresh Google Sheets tab, click cell A2, Ctrl+V to paste. Here's the trick — Sheets keeps every embedded URL when you paste rich text. So now column A is a mess of titles and timestamps and sidebar text, but the YouTube URLs are sitting inside every cell as hyperlinks.

[Hover one cell to show the embedded URL preview.]

Step two — we run a small Apps Script that walks column A, pulls the URL out of every cell, filters to only YouTube links, dedupes, and writes them back as clean plain text.

And here's where I want to show you the two ways to actually get this code.

[Cutaway TH or split-screen.]

The first way — what I do — is Claude inside Antigravity. I open my project, paste the prompt from the companion doc, and Claude writes the file straight into the folder. Done in about ten seconds.

[Show Antigravity with the prompt being pasted, file appearing.]

But — this is important — you don't need a paid AI subscription for any of this. The second way is, you take the same prompt, paste it into ChatGPT or Claude.ai or Gemini — whatever free thing you have access to — copy the code, save it yourself. The result is identical. Every prompt I'm using is in the description.

[Back to the Sheet. Apps Script editor. Paste, save, refresh.]

Drop the Apps Script into the Sheet — Extensions, Apps Script, paste, save. The Tools menu gets a new option called Clean YouTube URLs. Click it.

[Watch column A clean itself. Show the resulting list.]

Column A is now a clean, deduped list of YouTube URLs. File, Download, Comma Separated Values. CSV done.

[Cut to Finder showing the YT_Transcript_System folder structure.]

Now the pipeline itself. Three folders — input, output, tools — and one shell script. The CSV goes in the input folder, named whatever you want. I name mine after the creator. So this one's called nateherk.csv.

[Open Terminal.]

The shell script needs one tool — yt-dlp. On Mac it's literally one command:

```
brew install yt-dlp
```

Thirty seconds. Then we run the pipeline:

```
./YT_Transcript_System_tools/build_creator_transcript.sh
```

[Watch yt-dlp output scroll.]

Here's what's happening. For every URL in the CSV, the script calls yt-dlp with a specific set of flags — write auto subs, English only, skip download. It is not downloading audio. It is not running Whisper. It's just pulling YouTube's existing auto-captions, which are already there. Then a chunk of bash strips out all the timestamp tags, drops the headers, dedupes the line stutter that auto-captions always have, and smashes everything into one paragraph per video.

[Cutaway TH — slight tone shift, smile.]

Real honesty moment — when I first wrote this on Mac, I caught two bugs in my own script the first time I ran it. Last URL was being silently skipped. Paragraphs weren't ending with the right newline. I fixed them on camera. That's the deal here. You're watching this get built, not watching a polished course.

[Back to Terminal. Pipeline finishes. Open the output file.]

Done. Here's the output folder. One file per creator. Every transcript inside, structured by video number, title, and date.

[Open the full transcripts folder. Scroll past dozens of files.]

And just to scale this up — this is everything I built before posting a single video. Not because I was being clever. Because I was genuinely curious. And it turned out to be the most useful thing I've made for this channel so far.

[COMMUNITY PLUG] — embedded in pipeline section

[Quick cutaway TH.]

Quick one — every script you just saw, plus the actual AI prompts I used to generate them, are all linked in the description and in my free community. Takes thirty seconds to join, no paywall. It's just where I'm dropping the resources for these videos in one place.

[USE CASE 1: VOICE REFERENCE] — Hybrid (TH intro + LIVE demo) — ~290 words

[TH cutaway.]

Okay — three use cases. This is the part that actually matters because the data on its own is useless. What you do with it is everything.

Use case one — the voice reference. This is something I genuinely could not have done without the transcripts.

[Cut to screen — open the voice reference markdown file.]

If you take a creator's two hundred videos and you feed them all into Claude with a structured prompt, you can pull out their entire pattern — their hooks, their CTAs, their transitions, their pacing, their analogies, the words they use, the words they avoid. The output is a markdown reference doc that you keep open while you're writing your own scripts.

Here's the one I built for the creator I'm modeling structure-wise. Let me show you what's in it.

[Scroll through the voice reference doc — pause on Section 1: Hook Patterns table.]

There's a hook patterns section — every type of hook this guy uses, how often, with verbatim examples. There's a transitions section — every phrase he uses to move between sections. There's a CTA template — and this is wild, his CTA is almost word-for-word identical across like fifty videos. That's the kind of thing you only see when you've got the data laid out.

[TH cutaway.]

Now here's the important part — I'm not copying this guy. The voice reference is structural reference, not a script template. It's like studying a musician's song structures before you write your own songs. You're not copying the song. You're learning how songs are built.

The full prompts to generate one of these for any creator are in the companion doc. You give it the transcripts, you ask it the right questions, you get back a doc that's worth more than any thumbnail course on YouTube voice.

[USE CASE 2: TOPIC GAP — THE CHANNEL'S THESIS] — Hybrid — ~290 words

[TH cutaway.]

Use case two — and this one actually changed my plan for the channel.

[Cut to screen — Claude open with a fresh chat.]

So I took a sample of these transcripts and I dropped them into Claude with a really simple prompt — what does this creator make a lot of, and what's missing? What's underserved? What gaps exist in this space?

[Run the prompt. Show the output appearing.]

And the output came back with a list of saturated areas — tutorials, model comparisons, here's-how-to-build-X kind of content. The space is already full of that. Polished. Optimised. Done well.

[Highlight the "underserved" section in the output.]

But here's what was missing. There was almost no raw, day-by-day, here's-what-I-actually-tried content. Nobody is showing the full mess of starting an AI business from scratch — including the failed parts. The tutorials are polished. The income claims are big. But there's almost no honesty about what actually building this stuff looks like before it works.

[TH cutaway. Slow down. This is the thesis line.]

And that's the gap I'm filling. Because the more I looked at this data, the clearer it got — there's a lot of polish in this space and not a lot of honesty. I figured the only thing I could actually do better than people who've been doing this for years was be the one who shows everything raw. The bugs. The dead ends. The times I have no idea what I'm doing. So that's what this channel is.

If I hadn't run this analysis, I would've made the same generic AI tutorials everyone else makes. The transcripts gave me the data to choose a different lane — the only lane I had any business actually being in.

[USE CASE 3: SCRIPTING ASSISTANT] — Hybrid — ~283 words

[TH cutaway.]

Use case three — and this is the one I'm actually using right now to write the scripts for this channel.

[Cut to screen — Claude with the voice reference, sample transcripts, and a video brief loaded as context.]

Once you've got the voice reference and the transcripts, you can use them both as context when you're working with Claude on a script. You give it the voice reference. You give it a couple of sample transcripts. You give it your video brief. You tell it what your own voice rules are — what you sound like, what you don't want to sound like, what words to avoid. And then you ask it to draft.

[Show the draft output appearing — scroll through it.]

What you get back isn't a finished script. Let me be very clear about that. Claude is a great writer, but Claude trained on the internet sounds like everyone. When you give it specific examples from real videos that work, the output gets way closer to what you actually want — but it's still not your voice.

[TH cutaway. Personal tone.]

Honest moment — I rewrite a lot of what comes back. Sometimes most of it. The AI doesn't know me. It doesn't know what I find funny, what makes me roll my eyes, what I'd actually say out loud. It gives me a starting point. The starting point is way better than a blank page. But I still have to fight with it. I have to rewrite paragraphs. I have to tell it no. I have to tell it stop using the word leverage. And honestly, doing that fight is part of how I'm finding my own voice. The blank page never taught me anything about how I sound. Editing AI drafts has.

[WRAP + FRAMEWORK + NEXT-EPISODE TEASE] — TH, pre-recorded — ~155 words

[Same talking head shot as hook.]

So that's day one. Three steps. Collect — Sheets, Apps Script, one shell command. Analyse — voice reference, content gaps, structural patterns. Apply — scripting context, idea generation, channel positioning.

The thing I want you to take from this — you don't need years of experience to start. You need data and you need to be honest about where you actually are. That's it. The data part is now free. The honesty part is on you. And honestly, that's the harder one — most people who fail at this fail because they pretend to be further along than they are.

Next episode, I'm going to show you exactly how I'm using this system to write the script for the second video on this channel — including the parts where Claude gets it wrong and I have to fight with it. If you want to see that one, you know what to do.

[CTA] — TH, pre-recorded — ~64 words

If you got something out of this, or you're starting your own thing — give it a like. It genuinely helps. Every script and prompt is in the description, plus a free community where I'm dropping these resources as I go. If you want to see the rest of the journey, hit subscribe. Appreciate you sticking around — see you in the next one.

Total word count: ~1,938 words → ~12.9 min @ 150 wpm

Voice checklist — verified against draft v1

[x] Hook type: Personal-Moment / Build-in-Public, plants series flag in sentence 1

[x] Credential = lack of credential, journey-framed ("haven't built a successful AI business yet, that's the whole point")

[x] Framework pre-announced ("three steps") before delivery (in wrap)

[x] 3+ transitions identified: hookpipeline ("Let me show you why" "Okay so here's the punchline"), pipelineuse cases ("Okay — three use cases"), use caseswrap ("So that's day one")

[x] Musician/song structure analogy in Use Case 1

[x] Vocabulary check: no "audience" / no "content" (self-ref) / no "furthermore." "Subscribers" appears once, in journey-framed CTA only.

[x] Signature phrases used (3–4 max): "honestly" (×3), "real quick" / "quick one" (×2), "you guys" (×0 — could add 1–2 if delivery feels sterile in read-through), "let me show you" (×2)

[x] Specificity: 2,000 transcripts, "couple of weeks", real tool names (yt-dlp, Apps Script, Antigravity, Whisper-mentioned-as-NOT-needed), no Whisper-as-step

[x] Vulnerability landing points: credential ("haven't built a successful AI business yet"), demo ("two bugs in my own script"), Use Case 3 ("rewrite a lot of what comes back / fight with it")

[x] Series language present without overuse: "day one" (×3), "from scratch" (×4), "next episode" (×1), "journey" (×2)

[x] Episode 1 stands alone: someone watching this without context still gets a complete useful tutorial. The series framing is a multiplier, not a crutch.

Read-through notes for Daniel

Pacing in the demo section — the LIVE chunks should be quick. Don't pause to over-explain commands — just run them and narrate the punchline. The TH cutaways are where the explanation lives.

The "two bugs" beat — this is a key vulnerability moment. Lean into it. Smile. Don't apologise.

The Use Case 2 thesis line — "And that's the gap I'm filling" — slow down here. This is the line that defines the channel. Everything after this is downstream of this sentence.

The wrap framework — Collect / Analyse / Apply — say each one with a beat between them. Visual on screen if possible.

Optional "you guys" sprinkles — if the read-through feels too British-formal, add 1-2 "you guys" instances at natural beats (e.g. "Let me show you guys what's in it" in Use Case 1).

Total runtime risk — 1,938 words at 150 wpm = 12.9 min. Live demos always run longer than scripted talking head, so realistic landing is probably 14-15 min total. That's still in the target window. If post-edit is over 16 min, trim from the pipeline narration first (it has the most fluff).